

Rahnuma

रहनुमा رہنما

A working guide to the ILM, PANINI and Zistgah stack, from bytes and scripts to compilers, Pāṇini and quantum substrates

प्रकृति

	written	Devanagari hub	Romenagri	and back
Devanagari	प्रकृति	→ प्रकृति	→ prak_ri_ti	→ प्रकृति
Telugu	ప్రకృతి	→ प्रकृति	→ prak_ri_ti	→ ప్రకృతి
Bengali	প্রকৃতি	→ प्रकृति	→ prak_ri_ti	→ the same identity
Urdu, as usually written	ہندوی	→ हनदवी	→ _hana_dawee	→ हनदवी
Urdu, with zer and sukun	ہندوی	→ हिन्दवी	→ _hin_dawee	→ हिन्दवी

One identity, many scripts. Telugu and Bengali meet Devanagari in one hub, become the same Romenagri word, and come back exactly. Urdu meets the hub at what is written: without its short-vowel marks the word reads hanadawee; with zer and sukun it is hindawi. Every cell here was produced by the Hindawi tools while this guide was built.

Abhishek Choudhary, AyeAI. Version 2.0.0, 3 October 2026.

Rahnuma, the edition for the field: Information and Communication Technologies (ICTs)

Contents

Part I: Orientation 2

- 1. How to use this guide 3
- 3. Start here: find your way in 4

Part III: Setting up 6

- 13. Choosing a platform 7
- 14. The baseline environment and a first run 10
- 15. Running every component 14

Part IV: Foundations 26

- 19. Writing systems, sound, and reversible transliteration 27
- 21. Compilers, from lexer to microcode 34

Part V: Working with the estate 46

- 25. Open source, provenance and contributing 47

Appendices 49

- D. For researchers in science, technology, engineering and mathematics 50

Part I: Orientation

1. How to use this guide

In plain words. This guide explains a large body of work on computers, languages and intelligence. You can read it, run the programs on your own computer, or ask an AI to help you, and every example in it was really run.

This guide is for anyone who wants to use, check or extend the work gathered in the *zistgah*, *project-ilm* and *hindawiai* organisations on GitHub: students preparing for GATE, engineers who write compilers, firmware or chip layouts, linguists who work with scripts and sound, and researchers in quantum computing and AI. It asks for curiosity and a laptop, and nothing else.

There are four ways in.

- **Understand.** Part II gives the whole architecture first: where the work comes from, how the parts compose, each term with its nearest analogue and the difference, and the three Acts.
- **Read.** Parts IV and V explain the ideas the work rests on, with the mathematics written out: how numbers and letters sit in memory, what makes a transliteration reversible, how Pāṇini's grammar computes, how a compiler turns text into machine instructions, and how an oscillator network and a quantum circuit attack the same problem.
- **Run.** Part III gives the exact commands to set up Linux, Windows with WSL, or macOS, and to run every component of the estate on your own machine.
- **Ask.** Chapter 24 shows how to put any AI assistant to work on these repositories, and how to check what it tells you.

Conventions used throughout:

- Every example that could run while this guide was being built did run, and the text shows its real output, marked *Executed while this guide was built*. The package's own `gate`, `bash ops/verify.sh`, runs those examples again and fails if any output differs from what is printed here.
- Status words are used exactly. **Tested:** a test in the repository asserts it. **Executed:** it ran and its output is shown. **Written:** the code exists but has not been run here. **Mocked:** a stand-in replaces hardware that is not attached. **Planned:** described, not built.
- Everything said about the repositories was retrieved from the repositories themselves on 29 September 2026: their listings on GitHub, their `CITATION.cff` files and their READMEs. Where a repository does not record something, this guide says so instead of filling the gap.
- A dagger, †, marks a statement that follows the author's working notes and write-ups as supplied on 29 September and 3 October 2026, rather than the repositories.
- Sanskrit is transliterated in IAST. The spelling is British.

The work lives at github.com/zistgah, github.com/project-ilm and github.com/hindawiai, on the site zistgah.org, and in the Zenodo records listed in Chapter 12.

3. Start here: find your way in

In plain words. Four steps to find your part of the guide: who you are, your board or field, what you want to do, and your path with a PDF made for you.

This guide serves a school student and a senior researcher; a village health worker and an investor. Nobody needs all of it. Four steps find your part, on the site or with this page:

1. **Who you are.** A school student, a college student, a postgraduate or doctoral researcher; a teacher; an academic researcher; a working professional, someone in public service, policy or law, someone in community, NGO or village work, an artist, writer or maker; an industry partner or investor; or an AI agent working for a person.
2. **Your board or your field.** A school student chooses a board, ICSE, ISC, CBSE, IGCSE, IB, a state board or another; and a class. Everyone else chooses a field from the international standard classification of fields of study, ISCED-F 2013, the same one FAKIR's lattice uses. It places each subject where its own scholars would: political science is under social and behavioural sciences, not the humanities; law is under business, administration and law; linguistics is under languages.
3. **What you want to do.** Learn the basics, run the software, study or research, teach, contribute, or adopt, fund or invest.
4. **Your path and your edition.** The site lists your chapters in order, with reading times and the components to try first, and offers a PDF edition made for you. Every edition below is a printed selection from this one guide, so all of them stay in step.

Edition	PDF
School student	role-school.pdf
College student	role-college.pdf
Postgraduate or doctoral researcher	role-pg.pdf
Teacher or lecturer	role-teacher.pdf
Academic researcher	role-researcher.pdf
Working professional or engineer	role-professional.pdf
Public service, policy or law	role-public.pdf
Community, NGO or village work	role-community.pdf
Artist, writer or maker	role-creator.pdf
Industry partner, investor or funder	role-industry.pdf
Generic programmes and qualifications	field-00.pdf
Education	field-01.pdf
Arts and humanities	field-02.pdf
Social sciences, journalism and information	field-03.pdf

Edition	PDF
Business, administration and law	field-04.pdf
Natural sciences, mathematics and statistics	field-05.pdf
Information and Communication Technologies (ICTs)	field-06.pdf
Engineering, manufacturing and construction	field-07.pdf
Agriculture, forestry, fisheries and veterinary	field-08.pdf
Health and welfare	field-09.pdf
Services	field-10.pdf

The first public version of this guide, written for advanced researchers and senior professionals, stays as it was at [version 1](#).

Part III: Setting up

13. Choosing a platform

In plain words. Which computer set-up to use: Linux works best, Windows works through WSL, and a Mac works for most things.

Platform	What works	What is limited
Linux, native: Ubuntu 24.04 LTS or 26.04 LTS	Everything in the estate: shell scripts, compilers, Docker, GPU work, serial devices	Nothing; this is the reference platform
Windows 10 or 11 with WSL 2	Nearly everything, inside a real Ubuntu	USB devices need usbipd-win; the GPU is reached through the Windows driver; files under <code>/mnt/c</code> are slow
Windows, native	Reading, the browser-based tools, editing, Python, Node	The estate's bash scripts, Docker Engine, OpenTimestamps and systemd services do not run natively
macOS	Most command-line work, through Homebrew	No CUDA; BSD versions of the core tools unless GNU versions are installed

The estate's own workstation is a Ryzen 7 laptop with 16 GB of memory and an RTX 3050 with 4 GB, running Ubuntu with Docker; and TransEg names the same target. Any machine with 8 GB of memory and 30 GB of free disk runs every example in this guide.

Installing Ubuntu on a PC

1. Download the desktop image and the `SHA256SUMS` file from ubuntu.com/download/desktop into one folder.
2. Check the download. The command must print `OK` against the image's name:

```
sha256sum -c SHA256SUMS --ignore-missing
```

3. Write the image to a USB stick of 8 GB or more. On Windows, use Rufus or balenaEtcher. On Linux, find the stick first; the commands below assume it is `sdb`, and they erase it:

```
lsblk -d -o NAME,SIZE,MODEL
ISO=$(ls ubuntu-*-desktop-amd64.iso)
sudo dd if="$ISO" of=/dev/sdb bs=4M status=progress oflag=sync
```

4. Boot from the stick (the boot-menu key is usually F12, F10, F2 or Esc), choose *Install Ubuntu*, and either erase the disk or install alongside Windows. Before installing alongside Windows, turn off Windows Fast Startup, and if BitLocker is on, save its recovery key or suspend it. Secure Boot can stay on; Ubuntu's boot loader is signed.
5. After the first login:

```
sudo apt update && sudo apt full-upgrade -y
```

Windows with WSL 2

In a PowerShell window run as Administrator:

```
wsl --install
```

Restart, then:

```
wsl --list --online  
wsl --install -d Ubuntu-24.04  
wsl --set-default-version 2  
wsl --list --verbose
```

The last command must show `VERSION 2` beside Ubuntu. Open Ubuntu from the Start menu, create your Linux user and update it with `sudo apt update && sudo apt full-upgrade -y`. Then switch on systemd, which Docker Engine and several estate services expect:

```
printf '[boot]\nsystemd=true\n' | sudo tee /etc/wsl.conf
```

Run `wsl --shutdown` in PowerShell and open Ubuntu again; `systemctl is-system-running` now answers `running` or `degraded`.

Five rules that save hours:

- Keep your work in the Linux file system, under `~/work`, never under `/mnt/c`; file access across that boundary is many times slower. Windows reaches your Linux files at `\\wsl$`.
- Use VS Code on Windows with its WSL extension, and type `code .` inside a Linux folder.
- For an NVIDIA GPU install only the Windows driver; do not install a Linux display driver inside WSL. `nvidia-smi` inside Ubuntu then lists the card.
- To hand a USB device such as an ESP32 board to WSL, install `usbipd-win` with `winget install usbipd`, then in an Administrator PowerShell run `usbipd list`, `usbipd bind --busid 1-4` and `usbipd attach --wsl --busid 1-4`, using the bus identifier that `usbipd list` printed for your device in place of `1-4`.
- To cap WSL's memory, create a file named `.wslconfig` in your Windows user folder containing the two lines `[wsl2]` and `memory=8GB`.

Windows, native

```
winget install --id Git.Git -e  
winget install --id Python.Python.3.12 -e  
winget install --id OpenJS.NodeJS.LTS -e
```

```
winget install --id Microsoft.VisualStudioCode -e
git config --global core.autocrlf input
```

The estate's scripts are bash scripts written for GNU tools. Git Bash runs simple ones, but not systemd services, Docker Engine or OpenTimestamps workflows. flex and bison come through MSYS2, with `pacman -S --needed base-devel flex bison mingw-w64-ucrt-x86_64-gcc`. Windows file systems ignore case, lock open files and end lines with CRLF, and each of these breaks scripts written for Linux; the `autocrlf input` setting above prevents the third. Native Windows is right for reading, for the browser-based tools (the PANINI studio, CHAKRA, Jyotish, the Zamin simulator) and for Python notebooks. Build and run everything else in WSL.

macOS

```
xcode-select --install
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install git python@3.12 node flex bison gcc make
echo 'export PATH="$(brew --prefix bison)/bin:$(brew --prefix flex)/bin:$PATH"' >> ~/.zshrc
```

macOS ships a bison from 2006 and BSD versions of `sed`, `grep` and `date`. Homebrew installs flex and bison “keg-only”, that is, not on the PATH, and the last line puts them first. There is no CUDA on current Macs; Docker runs through Docker Desktop or Colima.

14. The baseline environment and a first run

In plain words. How to install the basic tools and run your first real program from this work.

Packages

On Ubuntu, native or in WSL:

```
sudo apt update
sudo apt install -y build-essential git curl wget unzip make cmake pkg-config \
    flex libfl-dev bison gawk gdb python3 python3-venv python3-pip pipx nodejs npm jq \
    fonts-noto-core fonts-indic
gcc --version | head -1; flex --version; bison --version | head -1
python3 --version; node --version
```

This brings GCC and binutils, the lexer and parser generators of Chapter 21 with flex's library and the gawk that Hindawi's driver needs, Python with virtual environments, Node for the JavaScript engines of PANINI, CHAKRA and Jyotish, and fonts for the Indic scripts.

Git and GitHub

Use the name and email of your own GitHub account in place of the example ones:

```
git config --global user.name "Ada Lovelace"
git config --global user.email "ada@example.org"
git config --global init.defaultBranch main
ssh-keygen -t ed25519 -C "ada@example.org"
cat ~/.ssh/id_ed25519.pub
```

Paste the printed key at GitHub under *Settings, SSH and GPG keys*, and check it with `ssh -T git@github.com`. The GitHub CLI turns forks and pull requests into single commands:

```
sudo apt install -y gh
gh auth login
```

Docker Engine

These are Docker's own commands for Ubuntu, from docs.docker.com/engine/install/ubuntu:

```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker "$USER"
```

Log out and back in, then `docker run hello-world` must greet you.

A first run: PANINIq

PANINIq, the quantum-simulation backend of PANINI, is a good first repository: small, tested, and with its own gate. Its README gives the install; the lines below follow it, with the virtual environment kept outside the clone:

```
mkdir -p ~/work && cd ~/work
git clone https://github.com/zistgah/paniniq.git
python3 -m venv ~/work/venv-paniniq
. ~/work/venv-paniniq/bin/activate
cd paniniq
export PYTHONDONTWRITEBYTECODE=1
pip install -r requirements.txt qiskit
python3 -m unittest discover -s tests
python3 demo.py
bash ops/verify.sh
```

The environment variable and the outside virtual environment matter because the repository's gate counts any `__pycache__` folder or compiled `.pyc` file inside the clone as a build artefact (its clause P08). Python writes those caches when it runs the tests, and a virtual environment inside the clone holds thousands of them, so either would turn the gate red for a reason that has nothing to do with the code.

This is the run made while this guide was built, on the repository at commit 82a5cf2:

Recorded run: `data/logs/paniniq-run-2026-09-29.txt`

```
$ git log -1 --format='%h %ad' --date=short
82a5cf2 2026-09-28

$ export PYTHONDONTWRITEBYTECODE=1

$ python3 -c 'import numpy, scipy, qiskit; print(numpy.__version__, scipy.__version__,
qiskit.__version__)'
2.5.3 1.18.1 2.5.2

$ python3 -m unittest discover -s tests
-----
Ran 23 tests in 0.217s

OK

$ bash ops/verify.sh
Panini Q: contract verification
PASS    P01  every source carries the copyright and SPDX lines
PASS    P02  no affiliation other than AyeAI is claimed
PASS    P03  the test suite passes
PASS    P04  the suite catches the anti-ferromagnetic zeroing bug
PASS    P05  demo.py runs to completion
PASS    P06  LICENSE is the FSF's verbatim GPL-3.0 text
```

And the demonstration, as it printed:

```
$ python3 demo.py
```

```
event 0: point=[0.85, 0.12, 0.44]
  order parameter R = 0.3874 |S| = 5  -> SPAWNED a node
event 1: point=[0.1, 0.9, 0.3]
  order parameter R = 0.5859 |S| = 5
event 2: point=[0.83, 0.14, 0.46]
  order parameter R = 0.4573 |S| = 6  -> SPAWNED a node
```

```
2. Lifecycle maintenance (decay priors, then prune/consolidate)
=====
|S| before maintenance: 8
pruned=1, merged=0
|S| after maintenance: 7
```

Quantum circuit diagram for CTQW(K, t) with 3 qubits. The circuit starts with q₀ in state |0>, q₁ in state |1>, and q₂ in state |2>. A dashed box labeled CTQW(K, t) encloses the first three qubits. After the dashed box, each qubit line has a measurement gate (M) and a classical control line. The classical control lines are labeled 0, 1, and 2. The measurement results are stored in a register labeled 'meas: 3/'. The circuit ends with a measurement gate (M) on each qubit line.

```
statevector dim = 8, sums to 1.000000 (unitarity check)
```

4. Max-Cut benchmark: oscillator vs. brute force vs. QAOA

```
oscillator    cut_value=4.0  time=2.90 ms
bruteforce    cut_value=4.0  time=0.22 ms
qaoa          cut_value=4.0  time=52.09 ms
```

5. Hardware bridge (mocked -- no real serial device exists here)

```
21 frame(s) 'sent' to MOCK
```

```
sample frame (hex): aa55000100b3
```

```
This only proves the K-matrix -> byte-frame packing is correct. No physical oscillator was touched --
there's no serial port or USB device available in this sandbox.
```

6. ClassicalSubstrate (PyTorch) scaling demo

Rahnuma 2.0.0

```
pedler/substrate.py::ClassicalSubstrate is written and ready -- `pip install torch` and re-run this script; it will automatically use a GPU if `torch.cuda.is_available()`.
```

Chapter 23 explains every number in it: the order parameter R , the spawned states, the continuous-time quantum walk on three qubits, and why the oscillator, brute force and QAOA agree on a cut of 4.

Every other component

Chapter 15 gives the commands for every component of the estate: the ones executed for this guide, and the rest as their own READMEs state them.

15. Running every component

In plain words. Step-by-step commands to start each part on your own computer, with what you should see when it works and what to do when it does not.

Every component below was cloned and run on a clean machine on 3 October 2026, with the commands its own README gives, and each card says exactly what happened. Where a step needed something the README does not mention, the card adds it; where a component did not pass, the card says why. Set up the machine first (Chapters 13 and 14).

Each card has the same shape: what the component is, what it needs, the commands to type in order, what you should see, and its status. Every card starts with a clone of the repository into a folder of the same name; the commands that follow are typed inside that folder.

Machine: Ubuntu 24.04, Python 3.12, Node 22, GCC 13. *Run on:* 3 October 2026.

Language and identity

Hindawi and Romenagri

What it is. The Hindawi Programming System's C front end, its driver hincc and the Romenagri script layer. *Needs:* gcc, make, flex, libfl-dev, gawk.

```
git clone https://github.com/hindawiai/chintamani.git && cd chintamani
mkdir -p .root/usr/bin .root/usr/lib .root/usr/include
make -C Romenagri all && make -C Romenagri install INSTRROOT="$PWD/.root"
export PATH="$PWD/.root/usr/bin:$PATH"
make -C Hindawi/guru all && make -C Hindawi/guru install INSTRROOT="$PWD/.root"
make -C Hindawi/hindr install INSTRROOT="$PWD/.root"
cp Hindawi/samples/HindiC.uhin . && hincc HindiC.uhin
printf 'रम\nरम\n' | ./hin.exe
```

You should see: संकलन के परिणाम, then the program asks आपका नाम क्या है? and counts to ten. *Status:* Executed for this guide. The compilers chapter prints the run.

The Urdu edition

What it is. Urdu shailis and the driver urducc, which sends an Urdu-script program through the Devanagari hub and Romenagri. *Needs:* the Hindawi build above, on the PATH.

```
git clone https://github.com/zistgah/urdu-ilm.git && cd urdu-ilm
mkdir -p .root/usr/bin .root/usr/lib .root/usr/include
make -C Romenagri all && make -C Romenagri install INSTRROOT="$PWD/.root"
export PATH="$PWD/.root/usr/bin:$PATH:<chintamani clone>/.root/usr/bin"
make -C ILM/guru all
bash ILM/urducc -s ILM/samples/UrduC.uhin
bash ILM/urducc ILM/samples/UrduC.uhin && ./hin.exe
```

You should see: C with the names na and ka, then the prompt in the hub script. *Status:* Executed for this guide. Replace with the folder of the Hindawi clone; the compilers chapter prints the run.

tajziya

What it is. The parser federation for the classical languages, by family. *Needs:* python3, make.

```
git clone https://github.com/zistgah/tajziya.git && cd tajziya
PYTHONPATH=src python3 -m tajziya commonalities
make check
```

You should see: make check: 19 passed, 0 failed, 1 unjudged. *Status:* Passed when run on 3 October 2026. Run the module with PYTHONPATH=src, from the clone's root.

The PANINI family

PANINI, the prompt-cycle language

What it is. The language the cyclers are written in, with its engine and studio. *Needs:* python3, node.

```
git clone https://github.com/zistgah/panini.git && cd panini
python3 panini.py check cyclers/pench.pni
python3 panini.py stages cyclers/yadein.pni
node tests/panini.test.mjs
python3 panini.py serve
```

You should see: 0 problems; 45 pass, 0 fail; the studio answers on 127.0.0.1:8717. *Status:* Passed when run on 3 October 2026.

PANINI, parser and interpreter

What it is. A general-purpose PANINI implementation with its conformance report. *Needs:* node.

```
git clone https://github.com/zistgah/panini_by_claude.git && cd panini_by_claude
node bin/panini.mjs conformance
npm test
```

You should see: 145 passed, 0 failed. *Status:* Passed when run on 3 October 2026.

PANINI, the stage-0 bootstrap

What it is. PANINI with a bootstrap that records its own evidence. *Needs:* node.

```
git clone https://github.com/zistgah/panini_by_grok.git && cd panini_by_grok
node src/cli.js run examples/hello.pni
node tests/run.mjs
node scripts/bootstrap.mjs
```

You should see: Hello, PANINI; 32 passed, 0 failed; stage-0 evidence written. *Status:* Passed when run on 3 October 2026. Its wider report, npm test, passes 81 of 83 here; the two others need tools this machine lacks.

The Humanesque merged release

What it is. PANINI, ILM, Hindawi, the cyclers and Mez in one tree. *Needs:* bash, node; optionally zig.

```
git clone https://github.com/zistgah/humanesque.git && cd humanesque
bash VERIFY.sh
npm test
```

You should see: 79 of 83 checks pass. *Status:* Partly passes, as run on 3 October 2026. The four that do not pass here need two downloadable model files (wllama.wasm and stories15M.Q4_0.gguf), the zig toolchain, and the Bengali flattening round trip.

PANINIq

What it is. The oscillator substrate, the PEDLER engine and quantum validation. *Needs:* python3, numpy, scipy, qiskit.

```
git clone https://github.com/zistgah/paniniq.git && cd paniniq
python3 -m venv ../venv-paniniq && . ../venv-paniniq/bin/activate
export PYTHONDONTWRITEBYTECODE=1
pip install -r requirements.txt qiskit
python3 -m unittest discover -s tests
python3 demo.py
bash ops/verify.sh
```

You should see: Ran 23 tests, OK; its gate 10 passed. *Status:* Executed for this guide. The first-run chapter prints the run.

PANINIB, biological realization

What it is. Biological intent lowered through typed IRs to a realization plan. *Needs:* python3.

```
git clone https://github.com/zistgah/paninib.git && cd paninib
python3 panini compile examples/replace.panini -o out
python3 panini inspect out --stage sequence-ir
python3 panini verify out
```

You should see: each step prints its JSON record. *Status:* Passed when run on 3 October 2026.

PANINIphy, physical realization

What it is. Physical intent compiled through a physical IR, with parts chosen by requirement. *Needs:* python3.

```
git clone https://github.com/zistgah/paniniphy.git && cd paniniphy
python3 panini compile examples/modular_block.panini -o out
python3 panini parts --category actuator --force 50N --stroke 20mm --voltage 12V
```

You should see: physical resolution=RESOLVED verify=True. *Status:* Passed when run on 3 October 2026.

Cyclers, desk and studio

matba, the press

What it is. Posters in, book out, sealed, pushed, minted. *Needs:* python3.

```
git clone https://github.com/zistgah/matba.git && cd matba
python3 matba.py serve
```

You should see: the press answers on 127.0.0.1:8710. *Status:* Passed when run on 3 October 2026.

khwab, the cutting room

What it is. The visual cycler. *Needs:* python3.

```
git clone https://github.com/zistgah/khwab.git && cd khwab
python3 khwab.py serve
```

You should see: the cutting room answers on 127.0.0.1:8711. *Status:* Passed when run on 3 October 2026.

awaz, the listening room

What it is. The audio cycler. *Needs:* python3.

```
git clone https://github.com/zistgah/awaz.git && cd awaz
python3 awaz.py serve
```

You should see: the listening room answers on 127.0.0.1:8712. *Status:* Passed when run on 3 October 2026.

tilasm, pench and yadein

What it is. The immersive, embodied and record cyclers; each repository has the same tests and its own page. *Needs:* node.

```
git clone https://github.com/zistgah/tilasm.git && cd tilasm
node tests/cycler.test.mjs
node tests/inbox.test.mjs
node tests/models.test.mjs
```

You should see: 20, 23 and 22 pass, 0 fail. *Status:* Partly passes, as run on 3 October 2026. Run the same three in zistgah/pench and zistgah/yadein. Two further test files, genie.test.mjs and leak.test.mjs, import modules the repositories do not carry.

GENIE

What it is. The Generalized Emotive-Narrative Interaction Engine. *Needs:* node.

```
git clone https://github.com/zistgah/genie.git && cd genie
node test/run.js
```

You should see: 191 passed, 0 failed. *Status:* Passed when run on 3 October 2026. Open index.html, or its page, for the engine itself.

alam

What it is. Small work, cycled with any AI. *Needs:* node, npm.

```
git clone https://github.com/zistgah/alam.git && cd alam
npm install jsdom
node test/alam-test.js
node test/dom-test.js
```

You should see: alam dom: 9 passed, 0 failed. *Status:* Passed when run on 3 October 2026. The page test needs jsdom, installed by the first line.

Mez, the Cognitive Workbench

What it is. The local-first desk. *Needs:* python3.

```
git clone https://github.com/zistgah/mez.git && cd mez
./mez doctor
./mez bearings
./mez serve
```

You should see: no account, no API key, no network required; the desk answers on 127.0.0.1:7373. *Status:* Passed when run on 3 October 2026.

Knowledge, domains and instruments

FAKIR

What it is. The knowledge kernel and its explorer. *Needs:* bash.

```
git clone https://github.com/zistgah/fakir.git && cd fakir
bash ops/verify.sh
```

You should see: CONTRACT SATISFIED, 6 checks. *Status:* Passed when run on 3 October 2026. The explorer is its page: zistgah.org/fakir

Dhancha

What it is. The domain spine every FAKIR domain is built on. *Needs:* bash, python3, make.

```
git clone https://github.com/zistgah/dhancha.git && cd dhancha
make check
```

You should see: site matches the registries; pr_gate bites on a claim and passes a clean change. *Status:* Passed when run on 3 October 2026.

ertabat

What it is. The communication-engineering domain. *Needs:* bash.

```
git clone https://github.com/zistgah/ertabat.git && cd ertabat
bash ops/verify.sh
```

You should see: CONTRACT OK. *Status:* Passed when run on 3 October 2026.

jugaad28

What it is. The 28 nm tape-out catalogue. *Needs:* bash.

```
git clone https://github.com/zistgah/jugaad28.git && cd jugaad28
bash ops/verify.sh
```

You should see: VERIFY GREEN. *Status:* Passed when run on 3 October 2026.

TransEg

What it is. The local digital-twin reference implementation. *Needs:* bash, python3; Docker for the stand-up.

```
git clone https://github.com/zistgah/transeg.git && cd transeg
./scripts/test.sh
./scripts/bootstrap.sh --dry-run
```

You should see: 10 passed; the stand-up plan. *Status:* Passed when run on 3 October 2026. The full stand-up, ./scripts/bootstrap.sh and docker compose, needs Docker and a GPU; it was not run here.

TransEg identity governance

What it is. Typed layers, envelope, staged policies and audit. *Needs:* python3, jsonschema, pytest.

```
git clone https://github.com/zistgah/transeg-idgov.git && cd transeg-idgov
pip install jsonschema pytest
python3 tools/idgov_validate.py policies
python3 -m pytest -q tests/
```

You should see: VALID: 8 stage policies; 14 passed. *Status:* Passed when run on 3 October 2026.

CHAKRA

What it is. The offline observatory and its calendars. *Needs:* node.

```
git clone https://github.com/project-ilm/chakra.git && cd chakra
node test/run.js
```

You should see: ALL SUITES PASSED. *Status:* Passed when run on 3 October 2026. Open index.html for the observatory.

Jyotish

What it is. The panchang, the Kaivalyik projection of CHAKRA. *Needs:* bash, node.

```
git clone https://github.com/zistgah/jyotish.git && cd jyotish
bash ops/verify.sh
```

You should see: PASSED, with 1 check skipped and named. *Status:* Passed when run on 3 October 2026. Open index.html for the panchang.

Cognition and hardware

PRATIK kernel

What it is. The balanced-ternary, event-driven kernel. *Needs:* g++; nvcc for CUDA.

```
git clone https://github.com/zistgah/pratik_core_mvp.git && cd pratik_core_mvp
bash build.sh
bash build.sh cuda
```

You should see: ALL CHECKS PASSED. *Status:* Passed when run on 3 October 2026. The CPU build passed here; the CUDA build needs nvcc and an NVIDIA GPU. The script is not marked executable, so call it with bash.

Zamin

What it is. Ternary silicon: SPICE cells, layout and a simulator. *Needs:* ngspice, python3.

```
git clone https://github.com/zistgah/zamin.git && cd zamin
cd spice && ngspice -b sign_frustration_tb.sp
```

You should see: a SPICE run of the ternary cells. *Status:* Does not run as is, as run on 3 October 2026. On ngspice 42 the testbench stops: the source v_gnd is reported as a shorted voltage source. The interactive simulator is its page.

QEDLER

What it is. The event-hypergraph physics programme. *Needs:* bash.

```
git clone https://github.com/project-ilm/qedler.git && cd qedler
bash ops/verify.sh
bash ops/resume.sh && cat RESUME.md
```

You should see: 5 of 6 clauses pass. *Status:* Partly passes, as run on 3 October 2026. Clause C2, manifest completeness, fails: 15 files are not in MANIFEST.sha256. Resealing the manifest clears it.

Provenance, tools and labs

Misty DOI

What it is. DOI minting and packaging for Zenodo. *Needs:* python3, pip.

```
git clone https://github.com/project-ilm/misty-doi.git && cd misty-doi
pip install misty-doi
misty --help
misty init
misty validate -m misty.json
```

You should see: `usage: misty [-h] [--version] {init,validate,transform,package,ots,publish}`. *Status:* Passed when run on 3 October 2026. Publishing needs a Zenodo token in ZENODO_TOKEN.

Research Kundali

What it is. A research map of a person, a lab or a paper. *Needs:* python3 and network access to the public records it reads.

```
git clone https://github.com/project-ilm/research-kundali.git && cd research-kundali
python3 kundali/kundali.py <ORCID, lab name or DOI> --out out
```

You should see: a research map in out/. *Status:* Not run here: needs network access. It reads public records such as ORCID; it could not reach them from this build machine, and stops rather than write an empty chart.

The linguistics lab

What it is. A starter lab in a container. *Needs:* Docker.

```
git clone https://github.com/project-ilm/linguistics-labs.git && cd linguistics-labs
docker build -t ilm-lab .
docker run --rm -p 8888:8888 -v "$PWD":/lab ilm-lab
```

You should see: the lab on localhost:8888. *Status:* Not run here: needs Docker. Not run here: this build machine has no Docker.

This guide

What it is. The site, the PDF and the Markdown, with their gate. *Needs:* python3, gcc, make, flex, libfl-dev, gawk, gdb; for a rebuild also pandoc, node, graphviz.

```
git clone https://github.com/zistgah/rahnuma.git && cd rahnuma
bash ops/verify.sh
bash examples/hindawi.sh words
python3 tools/build.py
```

You should see: 12 passed, 0 failed, 0 unjudged. *Status:* Passed when run on 3 October 2026.

Components you open in a browser

Component	Page
FAKIR's explorer	zistgah.org/fakir
The shared dome	zistgah.github.io/dome
AAB, the factory process	zistgah.org/aab
GENIE	zistgah.github.io/genie

tilasm, pench and yadein | zistgah.github.io/tilasm |
Kitab | zistgah.github.io/kitab |
Tok DOI | project-ilm.github.io/tok-doi |
CHAKRA | project-ilm.github.io/chakra |
Jyotish | zistgah.github.io/jyotish |
ILM | ilm.codes |
PANINI's studio | zistgah.github.io/panini |
The estate | zistgah.org |

Part IV: Foundations

19. Writing systems, sound, and reversible transliteration

In plain words. How Romenagri turns Indian scripts into plain English letters and back again without losing anything, and why Urdu written without its vowel marks is harder.

Kinds of writing system

- An **alphabet** writes consonants and vowels as separate letters of equal standing: Latin, Greek, Cyrillic.
- An **abjad** writes consonants, with vowels left out or added as optional marks: Arabic, Hebrew, and the Perso-Arabic scripts of Urdu, Sindhi, Kashmiri and Punjabi in Shahmukhi.
- An **abugida** writes a consonant with an inherent vowel; other vowels are marks on the consonant, and a virama removes the vowel: Brahmi and all its descendants, Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Tamil, Telugu, Kannada, Malayalam and more, and also Ethiopic.
- A **syllabary** has one sign per syllable: Japanese kana.
- A **logography** writes words and morphemes: Chinese characters. Egyptian hieroglyphs mix logograms with signs for sounds.

The Brahmi abugidas record the phonology of their languages closely, place of articulation by place of articulation, and that is why they can carry a whole computing stack: a mapping between their letters and a small ASCII alphabet can be made reversible, and the reversibility can be proved (below). In this work, full reversibility has so far been reached for the Brahmi family; for Latin written with diacritics, and for abjads that leave vowels unwritten, it has not yet been reached.

Script, language, standard: the three axes

ILM keeps three things apart.

Axis	What it is	Example
Script	How the text is written; handled by transliteration tables	Punjabi in Gurmukhi and in Shahmukhi: one language, two scripts
Language	The words: the vocabulary of keywords and names	Devanagari for Hindi, Marathi, Nepali and Sanskrit: one script, four languages
Standard	The computational construct and its host realisation	the C11 and C17 standards; a <code>for</code> loop in C, Python and Rust

A programming system that confuses these axes cannot grow past its first language. The pipeline in Chapter 21 applies them in order: script first, then language, then standard.

One spelling, several pronunciations

The consonant letter carries an inherent vowel, but languages pronounce it differently and drop it in different places.

Language	Inherent vowel	कमल (lotus) is said	Notes
Hindi, as spoken in Uttar Pradesh	a schwa, [ə]	kamal	the final inherent vowel is deleted, and some medial ones
Marathi	a schwa, [ə]	kamal	final deletion, as in Hindi
Bengali	an open o, [ɔ], raised to [o] before a high vowel in the next syllable	komol	the inherent vowel is usually kept inside a word and dropped at its end
Kannada	[a]	kamala	every vowel is pronounced; no deletion

The vowel written क, ṛ in IAST, is a second example. Hindi as spoken in Uttar Pradesh says it as *ri*; Marathi, Gujarati, Kannada and Telugu say it as *ru*. So प्रकृति is *prakṛiti* in Hindi and *prakṛuti* in Marathi and Kannada, and Bengali, which also says *ri* but rounds the inherent vowel, has *prokṛiti*. The three spellings pitri, pitru and pitra seen for पितृ record three spoken realisations of the same written vowel.

The consequence is exact: a Unicode string records the spelling, not the sound. A text-to-speech system, a transliterator or a keyword table has to know the language, not only the script, and the same Devanagari text read in Lucknow and in Pune is a different sequence of sounds.

Transliteration, transcription and romanisation

Transliteration maps letters to letters and can be made reversible. **Transcription** maps sounds to symbols, usually IPA, and is not reversible to a spelling. **Romanisation** is any rendering in Latin letters. IAST and ISO 15919 transliterate with diacritics (ṛ, ṣ, ṇ); Harvard-Kyoto and ITRANS use plain ASCII with capitals and letter pairs.

Romenagri

The Romenagri Transliteration System has been written since 2003 and released under the GNU GPL. Its current tree is the `Romenagri` folder of [hindawiai/chintamani](#), with a Perso-Arabic seed library in [project-ilm/romenagri](#). It is ASCII-7 and uses no diacritics. Its alphabet is exactly the characters a C identifier may contain apart from digits: A to Z, a to z and the underscore. The underscore is its one marker. It prefixes the independent vowels (अ is `_a`, आ is `_aa`); it separates the dental stops from the retroflex ones (त is `_ta`, ट is `ta`); it marks ह, the anusvara and the visarga (`_ha`, `ka_m`, `ka_hh`); and it keeps apart the readings that a plain Latin spelling would merge: ख is `kha`, क followed by ह is `ka_ha`, and क joined to ह by a virama is `k_ha`. A consonant written with no vowel after it joins the next consonant, as the virama does: क्ष is `kx`, and प्र begins `pr`.

Hindawi uses it through four filters. `uni2acii` turns Unicode into ACII, the 8-bit code, laid out after ISCII, in which the tools work; `acii2cf` writes ACII in Romenagri's compiler form, in which every word in the script becomes its Romenagri word while English words and the contents of string literals pass through untouched; `rmn2acii` and `acii2uni` go back. The output below was produced by those filters, built from the retrieved sources by `examples/hindawi.sh`, not typed in, and every entry was converted back and compared.

Executed while this guide was built: `bash examples/hindawi.sh words`

Letters, as Devanagari = Romenagri:

अ = _a	आ = _aa	इ = _i	ई = _ee	उ = _u	ऊ = _oo
ऋ = _ri	ए = _aee	ऐ = _ai	ओ = _oa	औ = _ou	क = ka
ख = kha	ग = ga	घ = gha	ङ = _nga	च = cha	छ = chha
ज = ja	झ = jha	ञ = _yna	ट = ta	ठ = tha	ड = da
ढ = dha	ण = _nna	त = _ta	थ = _tha	द = _da	ध = _dha
न = na	प = pa	फ = pha	ब = ba	भ = bha	म = ma
य = ya	र = ra	ल = la	व = wa	श = sha	ष = xa
स = sa	ह = _ha	ळ = lvra	क्र = kza	ज़ = jza	फ़ = phza
ड़ = _rda	ढ़ = _rdha	कं = ka_m	कः = ka_hh		

Names, to Romenagri and back through the inverse:

योग -> yoaga -> योग
जोड़ी -> joa_rdoa -> जोड़ी
सीमा -> seemaa -> सीमा
गिनती -> gina_tee -> गिनती
प्रकृति -> prak_ri_ti -> प्रकृति
पितृ -> pi_t_ri -> पितृ
क्षि -> kxi -> क्षि
ज्ञान -> j_ynaana -> ज्ञान
हृदय -> _h_ri_daya -> हृदय
उत्तर -> _u_t_tara -> उत्तर
क्षेत्रफल -> kxae_e_traphala -> क्षेत्रफल
त्रिकोण -> _trikoa_nna -> त्रिकोण
ख -> kha -> ख
कह -> ka_ha -> कह
कह -> k_ha -> कह

67 of 67 come back identical; every Romenagri form uses only A-Z, a-z and _: yes

ISO 15919, the international standard for transliterating Indic scripts, has a 7-bit fallback of its own, and Romenagri's tree records the correspondence in `iso15919_map.csv`. That fallback spells with punctuation:

Letter	ISO 15919	ISO 15919, 7-bit	Romenagri
ट	ṭ	.t	ta
त	t	t	_ta
ण	ṇ	.n	_nna
ञ	ṇ̐	~n	_yna
श	ś	;s	sha
ष	ṣ	.s	xa
ऋ	ṛ	,r	_ri

The first three columns are from `iso15919_map.csv`; the Romenagri column is the filters' output above. A full stop, a semicolon, a comma or a tilde cannot appear in a C identifier, an assembler symbol or an unquoted linker-script name, so the standard's own ASCII form cannot carry a program's names through a toolchain. Romenagri can, and that is why it exists.

One identity across scripts

ISCII gave every Brahmi script one layout, and Hindawi builds on that. Its `flatten_uni_dev`, generated from `iscii_map.csv`, maps each letter of Bengali, Assamese, Gurmukhi, Gujarati, Oriya, Tamil, Telugu, Kannada and Malayalam to its Devanagari counterpart. Devanagari is the hub; Romenagri runs on the hub; filters such as `fltr_hi_te` carry the hub back out to a script. The same word written in three scripts therefore has one identity, `prak_ri_ti`, which the cover of this guide shows.

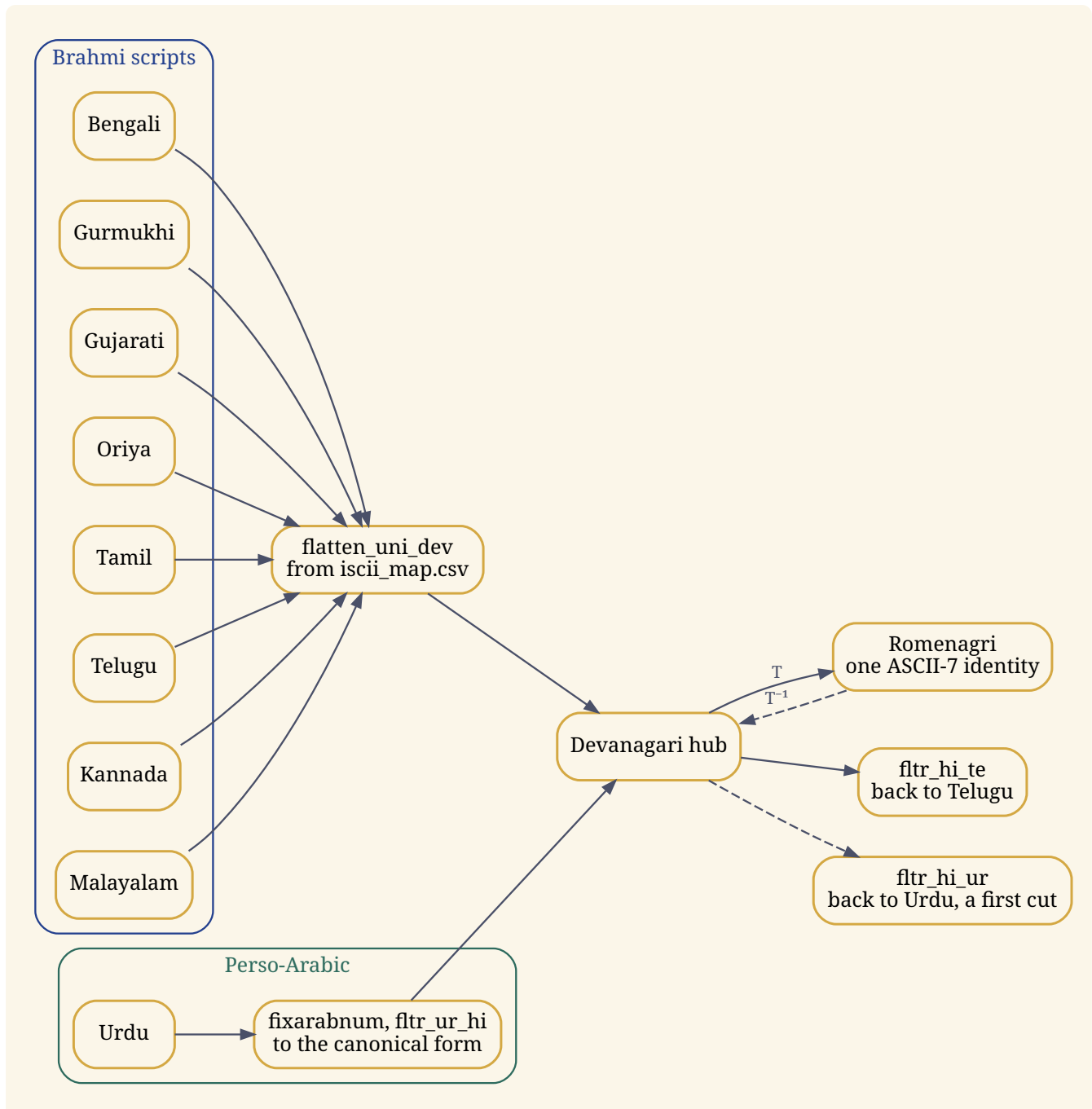


Figure 10. One identity across scripts: Brahmi scripts meet Devanagari in one hub; Perso-Arabic meets it at its written, canonical form.

This is the second pillar of linguistic equity at work. Because the script layer is independent of the language layer, any language can be written in any script, Hindi in Telugu letters or Telugu in Devanagari, and a program written either way is the same program to the compiler and the same names to the debugger.

Executed while this guide was built: `bash examples/hindawi.sh scripts`

1. The Brahmi hub: a word in Telugu or Bengali goes to Devanagari, to Romenagri, and back
 ప్రకృతి -> प्रकृति -> prak_ri_ti -> ప్రకృతి
 తెలుగు -> తెలుగు -> _taelugu -> తెలుగు
 গুণিত -> प्रकृति -> prak_ri_ti -> প্রকৃতি
 বাংলা -> বাংলা -> baa_mlaa -> বাংলা
2. Any language in any script
 Hindi written in Telugu script: हिंदी भाषा -> హిందీ భాషా
 Telugu written in Devanagari: తెలుగు భాష -> తెలుగు భాష
 Urdu written in Devanagari: ہندوی کتاب -> हन्दवी कताब
 the same, with zer and sukun: ہندوی کتاب -> हिन्दवी किताब
3. Measured on the corpora in this tree
 Hindi, words of letters and vowel signs: 984 distinct, 983 identical after Romenagri and back
 Telugu, through the hub and back: 1416 distinct, 1347 identical; of the 69 others, 69 carry the long-u sign U+0C42, which fltr_hi_te returns as U+0942
 Urdu: 1 of 794 words carry any zabar, zer, pesh, sukun, shadda or tanwin

Of the 984 distinct Hindi words made of letters and vowel signs, 983 come back identical; the exception is a malformed word in the corpus, with three u signs in a row, which comes back in its canonical form. Of the 1,416 Telugu words taken through the hub and back, 1,347 are identical, and every one of the other 69 carries the long-u sign ు, for which the reverse filter in this tree has no rule, so it returns the Devanagari sign: a one-line gap in `fltr_hi_te`, not in Romenagri. The author's own measurements in [project-ilm/romenagri](#) report 99.47 per cent of 5,807 words across the Hindi, Bengali and Telugu corpora exact. From the other side, over every string of up to four ASCII letters, his paper reports the 2003 to 2004 kernel as 98.68 per cent reversible or canonicalisable, with an irreducible floor of 1.31 per cent; the figure depends on the variant of the kernel measured, and his runbook says so.

The Perso-Arabic frontier

An abjad writes consonants and long vowels. The short vowels, zabar, zer and pesh, the absence of a vowel, sukun, and doubling, shadda, are marks that everyday Urdu leaves out: in the Urdu corpus in this tree, one word in 794 carries any of them. A written Urdu word therefore reaches the hub as what is written, its canonical form, and ہندوی arrives as हन्दवी, not हिंदवी. The mapping is exact and reversible to that form. What the script does not carry is which short vowel a speaker says.

With the marks written, the hub form is exact: ہندوی arrives as हिन्दवी, the author's own example in `hindawi_tashkil.txt`. So the difficulty is phonetic rather than a failure of the mapping. Which vowel a word carries has to come from knowledge of the language, its lexicon and its phonology, and that is precisely why ILM keeps the language axis apart from the script axis: the script layer maps what is written, exactly, and the language layer supplies what speakers know. The author estimates the words that need it at some 15 to 20 per cent.

Urdu programs already compile through the hub, as Chapter 21 shows. The reverse filter that renders hub text back into Urdu script, `fltr_hi_ur`, used by `urducc -r`, is a first cut in these trees; hardening the Perso-Arabic direction is the next step his seed release of Romenagri names. The Northwest Semitic abjads follow the same route to the Arabic hub: Hebrew final letter forms merge there, a documented many-to-one, and are reversible to the canonical form.

When is a transliteration reversible?

Let Σ be the source alphabet, the letters and marks of a script, and Γ the target alphabet, for example ASCII letters. A transliteration assigns each $\sigma \in \Sigma$ a non-empty target word $T(\sigma) \in \Gamma^+$ and extends to strings letter by letter:

$$T(\sigma_1\sigma_2\cdots\sigma_n) = T(\sigma_1)T(\sigma_2)\cdots T(\sigma_n).$$

It is reversible exactly when this extended map is injective, that is, when $T(s) = T(s')$ implies $s = s'$ for all strings $s, s' \in \Sigma^*$. Then there is an inverse with $T^{-1}(T(s)) = s$ for every string, which is the round-trip property a test can check. Injectivity holds if and only if T is one-to-one on single letters and the set of target words $C = T(\Sigma)$ is a *uniquely decodable code*.

Prefix codes. If no word of C is a prefix of another, every target string can be split into words from the left without looking ahead, so C is uniquely decodable.

Kraft and McMillan. If C is uniquely decodable over an alphabet of D symbols, with word lengths $\ell_1, \dots, \ell_m$, then

$$\sum_{i=1}^m D^{-\ell_i} \leq 1,$$

and whenever lengths satisfy this inequality, a prefix code with those lengths exists. A code whose sum exceeds 1 cannot be reversible, whatever else is done to it.

Sardinas and Patterson. For sets of words A and B write $A^{-1}B = \{w : aw \in B \text{ for some } a \in A\}$. Put $S_1 = C^{-1}C \setminus \{\varepsilon\}$ and $S_{k+1} = C^{-1}S_k \cup S_k^{-1}C$. The code is uniquely decodable if and only if no S_k contains a word of C . The sets are built from finitely many suffixes, so the test ends.

The example below runs both tests on four codes. The first is the mistake every naive scheme makes: `k` for `ک`, `h` for `ھ` and `kh` for `خ`. The string `kh` then reads both as `خ` and as `ک` followed by `ھ`, and the two tests catch it independently: the Kraft sum is $\frac{1}{2} + \frac{1}{2} + \frac{1}{4} = \frac{5}{4} > 1$, and the dangling suffix `h` is itself a codeword. The second is Romenagri's choice, as the output above shows: `ھ` carries the marker, `_h`, so the letter `h` never stands alone as a codeword and `kh` has exactly one reading.

Executed while this guide was built: `python3 examples/ud_check.py`

```
naive: k, h, kh          D=2  Kraft sum=5/4    (1.250)  uniquely decodable: False
Romenagri's choice: k, _h, kh  D=3  Kraft sum=5/9    (0.556)  uniquely decodable: True
not prefix-free: a, ab, bb    D=2  Kraft sum=1      (1.000)  uniquely decodable: True
prefix code: 0, 10, 110, 111 D=2  Kraft sum=1      (1.000)  uniquely decodable: True
```

```
"kh" under the naive code has 2 readings: k + h; kh
```

"khk" under the naive code has 2 readings: k + h + k; kh + k
"abbb" under a, ab, bb has 1 reading: ab + bb

The third code shows that a code can be uniquely decodable without being a prefix code; the price is lookahead, since **a** followed by **b** cannot be settled until the next symbols arrive. An abugida adds one more demand. KA with its inherent vowel, KA with a virama, and KA in a conjunct must all remain distinguishable in the target. Romenagri meets it by writing the inherent vowel (**ka**), by letting a consonant with no vowel after it stand for the virama (**k_ha** , **kx**), and by marking what would otherwise merge; and it keeps its whole alphabet inside the identifier alphabet, so that every transliterated name is at once a valid C identifier, a valid assembler symbol and a valid linker name. The domain of reversibility, whether all Unicode strings or only well-formed syllables, still has to be stated rather than assumed; the 67 entries above are the check this guide ran.

21. Compilers, from lexer to microcode

In plain words. How a program written in Hindi or Urdu becomes instructions a machine can run, with every name kept intact all the way to the debugger.

A compiler is a chain of translations, each into a representation closer to the machine: source text, tokens, a syntax tree, a checked tree, an intermediate representation, optimised intermediate code, assembly, an object file, an executable, a running process, and finally, inside the processor, micro-operations.

Lexing: regular languages and automata

Tokens are described by regular expressions. Thompson's construction turns a regular expression of length n into a nondeterministic finite automaton with at most $2n$ states; the subset construction turns that into a deterministic automaton whose states are sets of the original states, up to 2^n in the worst case and few in practice; Hopcroft's algorithm minimises the result in $O(n \log n)$ time.

lex (Lesk and Schmidt, Bell Labs, 1975) generates such a scanner in C from a list of patterns and actions; flex is the fast, free lex that Linux systems carry. flex works on bytes, so a keyword written in Devanagari is simply a pattern of UTF-8 bytes, and the whole Devanagari block is the pattern

`\xE0\xA4[\x80-\xBF]|\xE0\xA5[\x80-\xBF]`, whose end points the output in Chapter 18 printed.

Parsing: context-free grammars

A grammar $G = (N, \Sigma, P, S)$ has nonterminals, terminals, productions and a start symbol. The classic expression grammar is

$$\begin{aligned} E &\rightarrow T E' & E' &\rightarrow + T E' \mid \varepsilon \\ T &\rightarrow F T' & T' &\rightarrow * F T' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

A predictive, LL(1), parser chooses a production by looking at one token, guided by two families of sets. With $\neg$ marking the end of input:

$$\begin{aligned} \text{FIRST}(E) &= \text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \} \\ \text{FIRST}(E') &= \{ +, \varepsilon \}, \quad \text{FIRST}(T') = \{ *, \varepsilon \} \\ \text{FOLLOW}(E) &= \text{FOLLOW}(E') = \{), \neg \} \\ \text{FOLLOW}(T) &= \text{FOLLOW}(T') = \{ +,), \neg \} \\ \text{FOLLOW}(F) &= \{ +, *,), \neg \} \end{aligned}$$

LR parsing (Knuth, 1965) reads left to right, shifting tokens onto a stack and reducing them when a production's right side is on top, which builds a rightmost derivation in reverse. LALR(1) (DeRemer, 1969) merges LR(1) states that share a core, which keeps the tables small enough for practice. yacc (Johnson, Bell Labs) generates LALR(1) parsers in C, and GNU Bison is its compatible successor; the name is a joke on yacc, since a yak and a bison are cousins. When a grammar allows two moves, the generator reports a shift-reduce or reduce-reduce conflict, and declarations such as `%left '+' '-'`

and `%left '*' '/'` settle it by precedence and associativity: a rule-ordering device of the kind Pāṇini's 1.4.2 states for his grammar. The standard example is the dangling else, `if a then if b then s1 else s2`, which has two parse trees; yacc resolves it by shifting, so the `else` joins the nearest `if`.

Meaning: symbol tables and syntax-directed translation

After parsing, names are resolved through symbol tables, one per scope, and types are checked. An attribute grammar attaches values to the nodes of the tree: synthesised attributes flow up from children, inherited attributes flow down from parents and siblings, and definitions that use only synthesised attributes, or inherited attributes from the left, can be evaluated during parsing itself. This is what the GATE syllabus calls syntax-directed translation.

Intermediate representation

Three-address code has at most one operator per instruction:

```
t1 = x * 4
t2 = t1 + 6
return t2
```

Instructions without internal jumps form basic blocks, and the blocks form a control-flow graph. In static single assignment form every variable is assigned exactly once; where control paths join, a φ -function chooses the value from whichever predecessor was taken, $x_3 = \varphi(x_1, x_2)$. SSA is built by placing φ -functions at the dominance frontiers of the definitions (Cytron and colleagues, 1991). GCC's middle end works on GIMPLE in SSA form; LLVM's IR is SSA throughout.

Dataflow analysis as a fixed point

A variable is live at a point if its current value may still be read. For each instruction n ,

$$in[n] = use[n] \cup (out[n] \setminus def[n]), \quad out[n] = \bigcup_{s \in succ(n)} in[s].$$

Start with every set empty and apply the equations until nothing changes. The sets form a lattice ordered by inclusion, the equations are monotone, and the lattice has finite height, so the iteration stops at the least fixed point. Reaching definitions, available expressions and constant propagation are the same pattern over different lattices.

Optimisation, watched on a real compiler

The usual transformations are constant folding and propagation, common-subexpression elimination, dead-code elimination, moving loop-invariant code out of loops, strength reduction (a multiplication by four becomes a shift or an address computation), inlining, unrolling and vectorisation.

File: `examples/fold.c`

```

/* examples/fold.c: two small functions, to watch an optimising back end at work.
   © 1993-2026 Abhishek Choudhary. All rights reserved. AyeAI.
   SPDX-License-Identifier: GPL-3.0-or-later */
int scale(int x) { int k = 2 * 3; return x * 4 + k; }
int sum_to(int n) { int s = 0; for (int i = 1; i <= n; i++) s += i; return s; }

```

Captured while this guide was built: `gcc -O2 -S -masm=intel -fno-asynchronous-unwind-tables -fcf-protection=none -fno-ident -o - examples/fold.c | sed -n '/^scale:/,/size.sum_to/p'`

```

scale:
    lea eax, 6[0+rdi*4]
    ret
    .size    scale, .-scale
    .p2align 4
    .globl   sum_to
    .type    sum_to, @function
sum_to:
    test    edi, edi
    jle     .L6
    lea     ecx, 1[rdi]
    xor     edx, edx
    and     edi, 1
    mov     eax, 1
    je      .L5
    mov     eax, 2
    mov     edx, 1
    cmp     eax, ecx
    je      .L3
    .p2align 4,,10
    .p2align 3
.L5:
    lea     edx, 1[rdx+rax*2]
    add     eax, 2
    cmp     eax, ecx
    jne     .L5
.L3:
    mov     eax, edx
    ret
    .p2align 4,,10
    .p2align 3
.L6:
    xor     edx, edx
    mov     eax, edx
    ret
    .size    sum_to, .-sum_to

```

`scale` became a single instruction. `lea eax, 6[0+rdi*4]` computes $4x + 6$ in the address unit: the constant 2×3 was folded, and the multiplication by four was reduced to scaled addressing. In `sum_to`, GCC peeled one iteration when n is odd and then unrolled the loop by two, adding i and $i + 1$ in one step as $2i + 1$ with another `lea`. The exact output depends on the compiler's version and target; this one was produced by the GCC recorded in the package's `docs/BUILD.json`.

Registers, instructions and scheduling

Register allocation is graph colouring. Each variable's live range is a node; two nodes are joined when the variables are live at the same time; colouring the graph with k colours assigns k registers. Deciding whether a graph is k -colourable is NP-complete for $k \geq 3$, so compilers use heuristics: Chaitin's simplify-and-spill (1982), Briggs's optimistic colouring, and linear scan in just-in-time compilers where speed of compilation matters most. Instruction selection tiles the IR tree with machine instructions, instruction scheduling reorders them to hide latencies, and a peephole pass cleans up short sequences.

Assembling, linking, loading

The assembler writes an object file, ELF on Linux, with sections (`.text` , `.data` , `.rodata` , `.bss`), a symbol table, and relocations marking the places whose final addresses are not yet known. The linker resolves symbols across objects and libraries and applies the relocations; the dynamic loader maps shared libraries into the process when it starts.

Below the instruction set: micro-operations and microcode

On current high-performance processors, architectural instructions are decoded into internal micro-operations, and complex instructions are sequenced from microcode, the lowest layer at which code is still written by people; microcode can be updated after the chip is made. GATE's computer organisation syllabus asks for the design of hardwired and microprogrammed control units, and a microprogrammed control unit is exactly a small engine that runs microcode to drive the datapath.

Front ends and back ends: why a shared back end is not a dependency

With m source languages and n target machines, separate compilers would number mn ; with a shared intermediate representation, m front ends and n back ends suffice, $m + n$ in all. GCC's front ends for C, C++, Objective-C, Fortran, Ada, Go, D, Modula-2 and others all lower to GENERIC and GIMPLE and then RTL, and its back ends serve dozens of processors. LLVM's IR is shared by Clang, Rust, Swift, Julia and others. Nobody says that Rust depends on C++ because LLVM is written in C++: a new front end reusing a mature back end is the ordinary path.

The back end never sees keywords. By the time a program is GIMPLE or RTL, a loop written with `for` and the same loop written with `क्रम` have become the same structure: basic blocks, a condition and a branch. The human language of the keywords is settled and discarded at the front end. That is the precise answer to the objection that a system which uses GCC as its back end depends on English; the same holds for PANINIq's use of Qiskit, where what crosses the boundary is a Hamiltonian, a matrix of complex numbers. What the back end does see is the program's names, which it must carry into the object file intact; that is the script layer's job, and the last sections of this chapter run it.

Bootstrapping and the self-hosting fixed point

A compiler has three languages: the source S it reads, the target T it writes, and the language I it is written in, drawn as a T-shaped tombstone diagram. Diagrams chain: a compiler for X written in C, built by an existing C compiler; runs on the machine; with it you compile a compiler for X written in

X, and from then on the language compiles itself. The test of self-hosting is a fixed point: stage 1, built by the old compiler, builds stage 2; stage 2 builds stage 3; stages 2 and 3 must be identical. GCC’s own `make bootstrap` performs this three-stage build and compares the last two stages.

A reversible transliteration (Chapter 19) plays the part of the first compiler in such a chain. It lets the whole existing toolchain process programs written in the script, and its inverse restores the script in the output and the diagnostics, so the native-script tools can then be brought up the same way. Hindawi’s own keyword lexer was brought up exactly so, through Romenagri, as the next sections show.

Trust: the precise meaning of a sovereign stack

Ken Thompson showed in 1984 that a compiler can be altered to insert a back door into the login program, and to insert the same alteration into every compiler it later compiles, so that the back door survives after both source files have been cleaned. Reading the source is not enough. David Wheeler’s diverse double-compiling (2005, and his dissertation of 2009) answers it: compile the compiler’s source with a second, independently developed compiler; use the result to compile the source again, and compare. If the output is bit-for-bit identical to the original self-compiled binary, that binary corresponds to its source, under stated assumptions.

Together with reproducible builds, in which the same source always yields the same bits, this gives the phrase “sovereign stack” an exact technical meaning: a stack that its users can rebuild from source and verify with independent tools, with people able to do both. Which alphabet the keywords use has nothing to do with it.

Hindawi’s pipeline, run

The Hindawi Programming System has nine *shailis*, each a front end over one host language:

Shaili	Host
गुरु guru	C
श्रेणी shraeni	C++
यंत्र yantra	assembly
कृत्रिम ktritrima	Java
प्राथमिक praatha	BASIC
सूची soochee	Python
व्याकरण wyaaka	yacc
रोबोट robot	LOGO
शब्द shabda	lex

A program begins with a line naming its shaili, `<शैली गुरु>` for C, and one driver, `hincc`, compiles every shaili:

```
cat $1 | fixuninum | iconv -f utf-8 -t utf-16 | uni2acii | acii2cf | hincc.awk
```

`fixuninum` turns Devanagari numerals into ASCII digits; `iconv` and `uni2acii` turn Unicode into ACII; `acii2cf` writes the whole program in Romenagri's compiler form, every keyword and every name, leaving string literals as they are; and `hincc.awk` reads the shaili line, which is now `<shailee guru>`, and hands the rest of the program to that shaili's driver. For C that is `gurucc`:

```
cat $1 | acii2uni | iconv -f UTF-16 -t UTF-8 | h2c > tempfil0123.tmphin.c
gcc tempfil0123.tmphin.c -o hin.exe -lm
```

Its `acii2uni` restores the string literals to Unicode, so that the program prints in the script, and `h2c` maps each Romenagri keyword to its C keyword: `poor_nnaa_mka` (पूर्णक) to `int`, `krama` (क्रम) to `for`, `ma_likhoa` (म_लिखो) to `printf`, `mukhya` (मुख्य) to `main`. Every other word is a name, and it reaches GCC in Romenagri. The stages run in the order of the three axes: the script first, in `hincc`; then the language, in `h2c`, and for lex and yacc sources in `h2l` and its companions before it; then the standard, in flex and GCC.

The keyword lexer is itself written in Hindi. Its source, `h2c.uhin`, passes through the same `uni2acii` | `acii2cf` before flex compiles it, so the tool that reads Hindi programs is brought up through its own script layer, the bootstrapping step described above. The transducers also compose: a lex or yacc action block is C, so `h2l` carries only what lex adds beyond C, mapping `श_शब्द` to `yylex`, `श_ब_मान` to `yylval`, `श_माला` to `yytext` and `श_पंक्ति` to `yylineno`, and hands the rest to `h2c`. Counted that way, `shabda` has 6 rules of its own and 321 through `h2c`, 327 in all, and `wyaaka`, for yacc, has 341. Reading a composing shaili on its own would report a working system as broken.

The run below builds Romenagri, the guru shaili and `hincc` from the retrieved sources in `vendor/chintamani` with their own Makefiles, then compiles and runs Hindawi's own C sample:

Executed while this guide was built: `bash examples/hindawi.sh sample`

```
$ hincc HindiC.uhin
संकलन के परिणाम
=====

The C that gurucc handed to gcc:
#include <stdio.h>

int main()
{
    char _a[80];
    int ka;
    printf("आपका नाम क्या है?\n");
    scanf("%s", _a);
    printf("नमस्ते %s.\n", _a);
    for(ka=1; ka<=10; ka++)
        printf("%d\n", ka);
    scanf("%s", _a);
    return 0;
}

$ ./hin.exe, with राम typed at both prompts
आपका नाम क्या है?
नमस्ते राम.
```

```
1
2
3
4
5
6
7
8
9
10
```

अ has become `_a` and क has become `ka` ; the strings still print in Devanagari.

The symbol bridge, from source to debugger

This program, in the guru shaili, has a global, a function, a parameter and a local, all named in Hindi:

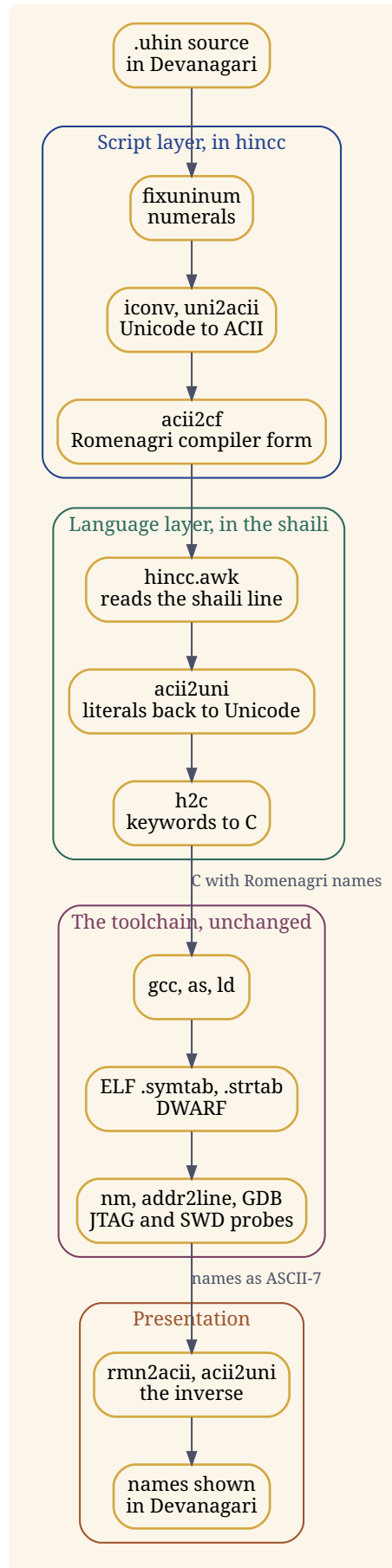


Figure 11. The symbol bridge: a Hindi program through hincc, GCC and the debugger; its names carried in Romenagri and rendered back.

File: examples/yog.uhin

```
<शैली गुरु>
#समावेश <मानकपन.स>

पूर्णक योग;

पूर्णक जोड़ो(पूर्णक सीमा)
{
    पूर्णांक गिनती;
    क्रम(गिनती=1; गिनती<=सीमा; गिनती++)
        योग = योग + गिनती;
    वापस योग;
}

पूर्णक मुख्य()
{
    म_लिखो("१ से १० तक का योग = %d\n", जोड़ो(10));
    वापस 0;
}
```

Executed while this guide was built: `bash examples/hindawi.sh yog`

```
$ hinc yog.uhin
संकलन के परिणाम
=====

The C that gurucc handed to gcc:
#include <stdio.h>

int yoaga;

int joa_rdoa(int seemaa)
{
    int gina_tee;
    for(gina_tee=1; gina_tee<=seemaa; gina_tee++)
        yoaga = yoaga + gina_tee;
    return yoaga;
}

int main()
{
    printf("1 से 10 तक का योग = %d\n", joa_rdoa(10));
    return 0;
}

$ ./hin.exe
1 से 10 तक का योग = 55
```

योग is `yoaga`, जोड़ो is `joa_rdoa`, सीमा is `seemaa` and गिनती is `gina_tee`: names over A to Z, a to z and the underscore, which GCC, the assembler, the linker and every later tool accept as they are. Below, the same program is built with debugging information and examined with the ordinary tools, and then the same transcript is rendered through the Romenagri inverse. The binary is not touched; only what a person reads changes. The C names that Hindawi has words for, `main` and `printf`, come back through `c2h` as they do in Hindawi's own `std2hin`; the program's own names come back through `rmn2acii` and `acii2uni`.

Captured while this guide was built: `bash examples/hindawi.sh debug`

```
$ nm --defined-only yog-g, the program's own symbols
0000000000001149 T joa_rdoa
0000000000001182 T main
0000000000004014 B yoaga

$ readelf --debug-dump=info yog-g, the names DWARF records
variable yoaga
subprogram printf
subprogram main
subprogram joa_rdoa
formal_parameter seemaa
variable gina_tee

$ gdb -batch: break joa_rdoa, run, backtrace, info args, finish, print yoaga
Breakpoint 1 at 0x1154: file ./tempfil0123.tmphin.c, line 8.

Breakpoint 1, joa_rdoa (seemaa=10) at ./tempfil0123.tmphin.c:8
8      for(gina_tee=1; gina_tee<=seemaa; gina_tee++)
#0  joa_rdoa (seemaa=10) at ./tempfil0123.tmphin.c:8
#1  0x000055555555194 in main () at ./tempfil0123.tmphin.c:15
seemaa = 10
0x000055555555194 in main () at ./tempfil0123.tmphin.c:15
15      printf("1 से 10 तक का योग = %d\n", joa_rdoa(10));
Value returned is $1 = 55
$2 = 55

$ addr2line -f -s -e yog-g 0x0000000000001149
joa_rdoa
tempfil0123.tmphin.c:6

The same, rendered through the Romenagri inverse; the binary is unchanged:
$ nm --defined-only yog-g, the program's own symbols
0000000000001149 T जोड़ो
0000000000001182 T मुख्य
0000000000004014 B योग

$ readelf --debug-dump=info yog-g, the names DWARF records
variable योग
subprogram म_लिखो
subprogram मुख्य
subprogram जोड़ो
formal_parameter सीमा
variable गिनती

$ gdb -batch: break जोड़ो, run, backtrace, info args, finish, print योग
Breakpoint 1 at 0x1154: file ./tempfil0123.tmphin.c, line 8.

Breakpoint 1, जोड़ो (सीमा=10) at ./tempfil0123.tmphin.c:8
8      for(गिनती=1; गिनती<=सीमा; गिनती++)
#0  जोड़ो (सीमा=10) at ./tempfil0123.tmphin.c:8
#1  0x000055555555194 in मुख्य () at ./tempfil0123.tmphin.c:15
सीमा = 10
0x000055555555194 in मुख्य () at ./tempfil0123.tmphin.c:15
15      म_लिखो("1 से 10 तक का योग = %d\n", जोड़ो(10));
Value returned is $1 = 55
$2 = 55

$ addr2line -f -s -e yog-g 0x0000000000001149
जोड़ो
tempfil0123.tmphin.c:6
```

Three layers do three jobs here. The front end lowers keywords to C, where they vanish into control flow. The symbol table and the DWARF information carry every name in Romenagri, byte for byte, through the assembler and the linker into the ELF file, and from there to `nm`, `addr2line`, GDB, OpenOCD on a JTAG or SWD probe, or a kernel's panic trace. A presentation layer applies the Romenagri inverse wherever a person reads: a GDB front end, an IDE panel, a filter on `addr2line` or on a panic trace. No link in the chain has to accept anything but letters, digits and the underscore, and nothing is lost, because the map is bijective. Drop the script layer and pass names as raw UTF-8, or flatten them into names that cannot be mapped back, and some link in that chain either rejects them or hands the developer names that no longer match the source.

Two details are worth knowing. GDB lists the generated C because the guru shaili does not emit `#line` directives; with them, the same session would list the Hindi source line by line. And Romenagri writes an independent vowel with a leading underscore, so a vowel-initial name at file scope, `_aapakaa` for `आपका`, begins with an underscore, which the C standard reserves for the implementation at that scope; GCC accepts it, and a project that exports such symbols can fix a prefix for them.

The Urdu edition, run

`zistgah/urdu-ilm` carries the same shailis for Urdu. The guru shaili's lexer is written in Urdu, `h2c_urdu.uhin`, and the driver `urducc` sends an Urdu program through `fixuninum` and `fixarabnum` for the digits, `fltr_ur_hi` to the Devanagari hub, `uni2acii` and `acii2cf` to Romenagri, then `h2c_urdu` and the standard Hindawi driver. The run below compiles the Urdu edition's own C sample:

File: `vendor/urdu-ilm/ILM/samples/UrduC.uhin`

```
<شیلی گرو>
#شامل <معیاری.س>
() صحیح مرکزی
{
    حرف ن[80];
    صحیح ک
    لکھو ("آپ کا نام کیا ہے؟\n");
    لکھو ("س", ن);
    لکھو ("س.\n", ن);
    براءے (ک=1; ک>=10; ++ک)
    لکھو ("د\n", ک);
    واپس 0
}
```

Executed while this guide was built: `bash examples/hindawi.sh urdu`

```
$ urducc UrduC.uhin
संकलन के परिणाम
=====

The C that reached gcc:
#include <stdio.h>
int main()
{
    char na[80];
    int ka;
    printf("आप का नाम क्या है?\n");
    scanf("%s", na);
```

```

printf("इलसलाम एलीकम %s.\n",na);
for(ka=1; ka<=10; ka++)
    printf("%d\n",ka);
return 0;
}

$ ./hin.exe, with Ali typed at the prompt
आप का नाम क्या है?
इलसलाम एलीकम Ali.
1
2
3
4
5
6
7
8
9
10

```

The keywords are C and the names are Romenagri, reached through the hub: ن is na and ک is ka. The strings print in the hub script and show the frontier of Chapter 19 at work: کیا, written without its short vowel, arrives as क्या rather than क्या, and ہے as है rather than है. Rendering them back into Urdu is the reverse filter's job.

ILM's construct model

In the merged release, `ilm/constructs.csv` holds 39 constructs in 27 human languages and `ilm/decorators.csv` holds 216 realisations of them in 8 host languages; a keyword table of 1,801 constructs in 27 languages generates the Hindawi distributions. The construct is primary: a counted loop is translated once per human language, not once per host language, and a decorator records what a host requires, such as `stdio.h` for printing in C. The construct model is the keyword layer, the layer `h2c` implements for C. Names still pass through the script layer, Romenagri, exactly as above.

Part V: Working with the estate

25. Open source, provenance and contributing

In plain words. How to share, copy and improve this work in the open, and how each piece is stamped with its date and a permanent reference.

Git as a data structure

A Git repository is a directed acyclic graph of commits. Each commit names a tree, the hash of the directory's contents, its parent commits, its author and its message, and every object is addressed by the hash of its content. Change one byte anywhere and every hash above it changes, which is why a commit hash pins an exact state of every file. A branch is a movable name for a commit; a merge commit has two parents.

Contributing on GitHub

```
gh repo fork zistgah/paniniq --clone
cd paniniq
git switch -c order-parameter-bounds
bash ops/verify.sh
git commit -am "Test that the order parameter stays within [0, 1]"
git push -u origin order-parameter-bounds
gh pr create --fill
```

Between switching branches and verifying comes your change. `gh repo fork --clone` also adds the original repository as a remote named `upstream`, so `git fetch upstream && git rebase upstream/main` brings your branch up to date. To report a problem instead of fixing it, `gh issue create --repo zistgah/paniniq` opens an issue.

Licences

The GNU GPL, version 3, is copyleft: anyone may use, study, change and share the software, and whoever distributes a changed version must distribute its source under the same terms. Permissive licences such as MIT and Apache 2.0 allow changes to be kept closed; Apache adds an explicit patent licence. Creative Commons Attribution-ShareAlike 4.0 does for text and media what the GPL does for code. Among the zistgah and project-ilm repositories, every licence that GitHub recognises is a version of the GPL; others carry licence files that GitHub does not recognise, or none that it detects. In every case the repository's own licence file, where it has one, is the authority. This guide's text is CC BY-SA 4.0 and its scripts are GPL-3.0-or-later.

White-labelling, and the commons

White-labelling is selling a product made by one party under another's brand. Under a permissive licence a company can take open code, change it and sell it closed; under the GPL it may still sell it, but it must pass on the source and the same freedoms. Contributing upstream is the opposite of white-labelling: the improvement returns to everyone.

The Aṣṭādhyāyī's review tradition is an old form of the same practice. Kātyāyana's vārttikas proposed amendments to specific rules, Patañjali's Mahābhāṣya discussed and adjudicated them, and the rules were kept intact alongside the commentary, the way a pull request and its review thread are kept alongside the code.

DOIs

A DOI is a persistent identifier that doi.org resolves to a landing page. Zenodo, run by CERN, issues DOIs through DataCite and keeps two kinds: a concept DOI that always points to the latest version, and a version DOI for each release. project-ilm/misty-doi writes the metadata and makes the deposit; its commands are in Chapter 14.

Timestamps with OpenTimestamps

A timestamp proves that a file existed before a certain time without revealing the file. The file's SHA-256 hash is one leaf among many; pairs of hashes are hashed together up a Merkle tree,

$$h_{\text{parent}} = \text{SHA256}(h_{\text{left}} \parallel h_{\text{right}}),$$

and the root is written into a Bitcoin transaction. The proof for one file is the path of sibling hashes from its leaf to the root, $\lceil \log_2 N \rceil$ of them for N files, together with the block that contains the root. Anyone can recompute the path and check that the file existed before that block was mined. The commands are `ots stamp FILE`, then `ots upgrade FILE.ots` once the block is confirmed, and `ots verify FILE.ots`.

Attestations

An in-toto statement is a signed JSON document that binds a subject, file names with their digests, to a predicate saying what was done to them: built, tested, reviewed. It travels in a DSSE envelope, and the SLSA framework defines levels of supply-chain assurance built from such statements. Candor records the estate's typed intents in this form, under each repository's `attest/` folder.

How the estate releases

Every repository carries a `CONTRACT.md` of numbered clauses and an `ops/verify.sh` that checks them and reports PASS, FAIL or UNJUDGED for each. A push happens only when the gate is green, and it writes a receipt, a timestamp, a manifest of hashes in `MANIFEST.sha256` and a seal; a mint then records the DOI in `CITATION.cff` and in the README. The governance repository's contract sets twelve clauses for all of this: sole authorship and AyeAI as the only affiliation; the copyright line; no fabrication; verification by execution; reading before writing; typed gates before anything irreversible, whether a mint, a push, a deletion, an issue or a pull request; staying inside the working folder; provenance travelling with every publication; fork, pull request and merge for shared repositories; small reviewable increments; an agreed split of roles between AIs; and declared repository classes. Clauses C31 to C37 add retrieval before assumption, no vendor lock-in, experience that scales with the user, certification in isolation and in integration, context that travels with every surface, routing as a preference rather than a property, and never bluffing. This package follows the same pattern; its contract is in `CONTRACT.md` and its gate is `bash ops/verify.sh`.

Appendices

D. For researchers in science, technology, engineering and mathematics

In plain words. A guide for researchers in science, technology, engineering and mathematics: what can be read, run and tested now, and what is still open.

Each field below lists what in the estate can be read, run and tested now, and what is open. Status words keep their meaning from Chapter 1: tested, executed, written, mocked, planned.

Computer science

Programming languages and compilers. Hindawi's composed transducers over unmodified host toolchains; the Romenagri symbol bridge through ELF, DWARF and GDB; the PANINI construct model, with 39 constructs in 27 languages and 216 decorators across 8 host languages; the bootstrap and its self-hosting fixed point (Chapters 21 and 6). Executed here. **Open:** `std2hin` doubles every underscore, so vowel-initial names do not round-trip back from C; the guru shaili emits no `#line` directives; the YACC shaili awaits a fresh conformance run.

Systems. The hinlin kernel tree and the mass conversion of real C sources, including kernel trace code, into Hindi and back (`hindawiai/chintamani`, `Romenagri/mass_hindawi`).

AI and machine learning. The cyclers and GENIE as AI-agnostic, human-inspected method (Chapter 8); Romenagri as a tokenisation substrate, where bytes fall 1.76 times and the alphabet from 51 to 24 symbols without loss, but token counts did not fall at matched merges, so the predicted gain is still to be shown; COPA's Authority Projection Index as a measurable property of a model's behaviour

Software engineering. Executable contracts and gates in every repository; the provenance chain, Tok DOI, spiguard, Candor and Misty DOI; VGC; the component cards of Chapter 15, each the result of a real run.

Data. ILM's registry of 7,867 languages and 226 scripts; 13,668 keyword standards; the corpora in the Romenagri tree.

Electrical and electronic engineering

The open chip-design flow and the 28 nm tape-out catalogue, jugaad28, with its evidence-gated descriptors (Chapter 22); Zamin's ternary cells in SPICE, whose testbench needs attention on ngspice 42 (Chapter 15); the PRATIK kernel, whose CPU build passes and whose CUDA build needs an NVIDIA toolchain; device control on ESP32, zasab, built but not yet pushed.

Physics

QEDLER, the event-hypergraph research programme for physics, self-declared as a framework with open problems; oscillator Ising machines and Kuramoto dynamics in PANINIQ, with their Lyapunov function (Chapter 23); superconducting qubits and the physics of the cold (Chapter 22).

Mathematics

Coding theory for reversible transliteration: Kraft and McMillan, and Sardinas and Patterson (Chapter 19); formal languages and rewriting in Pāṇini's grammar, with pratyāhāras as interval classes over an ordered list (Chapter 20); graph theory in the typed hypergraph and in Max-Cut; dynamical systems in the order parameter and in the oscillators' energy; the formal tuples of the CEM kernel, $\mathcal{E} = (E, \mathcal{C}, \Pi, W)$, and of the AyeAI Triad.

Biology and medicine

PANINIB, the biological realization arm, which lowers intent to sequence IR and claims no wet-laboratory result; CEMb, operated through Interglial Healthcare; the Quantum Neuromorphic BCI programme, with brain-to-text, cognitive prosthetics and intelligent rehabilitation as directions needing clinical collaboration; VIKRAM's Health section and its tools for an accessible virtual hospital; the earlier HMSEI wearable and the Dr Rho telepresence platform.

Cognitive science and neuroscience

PEDLER's two branches, through LVF and MLCNE to the cognitive enablement modules, and through NI2A2 to ANGEL; its primitives; the brain-computer interface work of Act III.

Astronomy and the earth and space sciences

CHAKRA, the offline observatory and calendars, whose JavaScript and C99 versions produce byte-identical output; Jyotish, derived from it; the habitats, with a lander embodiment on the Moon and a quadrotor on Mars inside the shared dome.

Robotics and manufacturing

PANINIphy, the physical realization arm, which compiles physical intent and chooses parts by force, stroke and voltage; pench, the embodied cycler; † the modular self-reconfiguring robots of the notes.

© 1993–2026 Abhishek Choudhary. All rights reserved. AyeAI. Text CC BY-SA 4.0; scripts GPL-3.0-or-later. Version 2.0.0, 3 October 2026.